Developer Portfolio



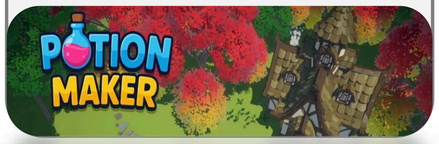
박우영
wyp1226@naver.com

INDEX

PORTFOLIO 1 >
Project : T



PORTFOLIO 2 >
Potion Maker



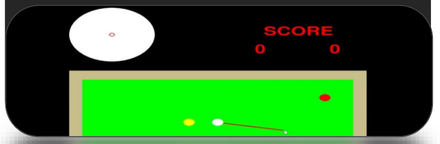
PORTFOLIO 3 >
SKYFALL : Resistance



PORTFOLIO 4 >
Last Flame Keepers



PORTFOLIO 5 >
Billiards 3 Ball : Pygame





Project : T

프로젝트 기간 | 2025-09-01 ~

프로젝트 인원 | 5 인 (기획 1, 클라이언트 3, 아트 1)

설명 | 용병을 데리고 돌아다니며 던전을 클리어하는 게임

담당 역할 | 게임 클라이언트 개발자, 모델링, 캐릭터 애니메이션

기술 스택 | Unity, C#, UniTask

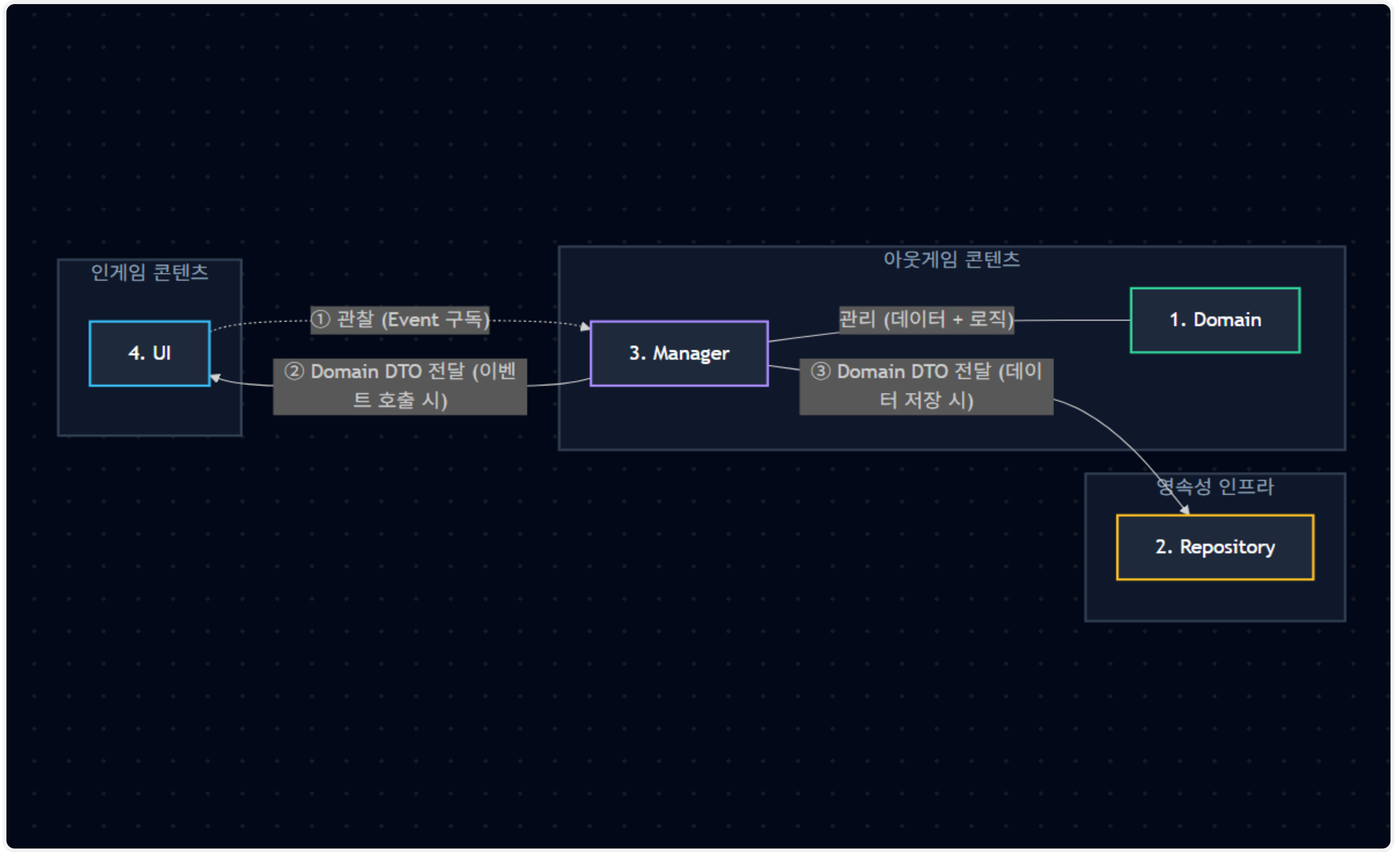
아웃게임 아키텍처 설계 >>>

도메인 주도 설계(DDD) 및 계층 분리

가독성을 고려한 명시적 데이터 구조화

세부 설명

- 4개의 레이어로 분리하여 관리
  - Domain, Repository, Manager, UI 레이어
  - 하위 레이어는 상위 레이어를 모르게 구현
- 데이터의 무결성 구조



DDD 클린 아키텍처 구조 도식화

Google Spreadsheet 연동 >>>

세부 설명

- 구글 클라우드 콘솔 및 Spreadsheet API를 활용한 데이터 파이프라인 구축
- 프로그래머 개입 없이 기획자가 직접 밸런스 데이터 수정 가능
- 데이터 주도형(Data-Driven) 설계로 협업 효율 극대화

MasterDataStore >>>

세부 설명

- 연동된 시트 데이터를 순수 C# 객체(POCO) 기반 Singleton으로 관리
- Unity API에 종속되지 않는 가벼운 컨테이너로 메모리 접근 속도 최적화
- 전역에서 안전하게 참조 가능한 중앙 집중형 읽기 전용(Read-Only) 구조 확립

UniTask 비동기 처리 >>>

세부 설명

- Coroutine의 GC 할당 문제 및 반환값 문제를 해결하기 위해 UniTask 전면 도입
- 대규모 데이터 파싱 및 로딩 작업을 직관적이고 안전한 비동기로 제어
- 데이터 로드 및 UI 연출 시 메인 스레드 병목을 방지하여 프레임 드랍 차단

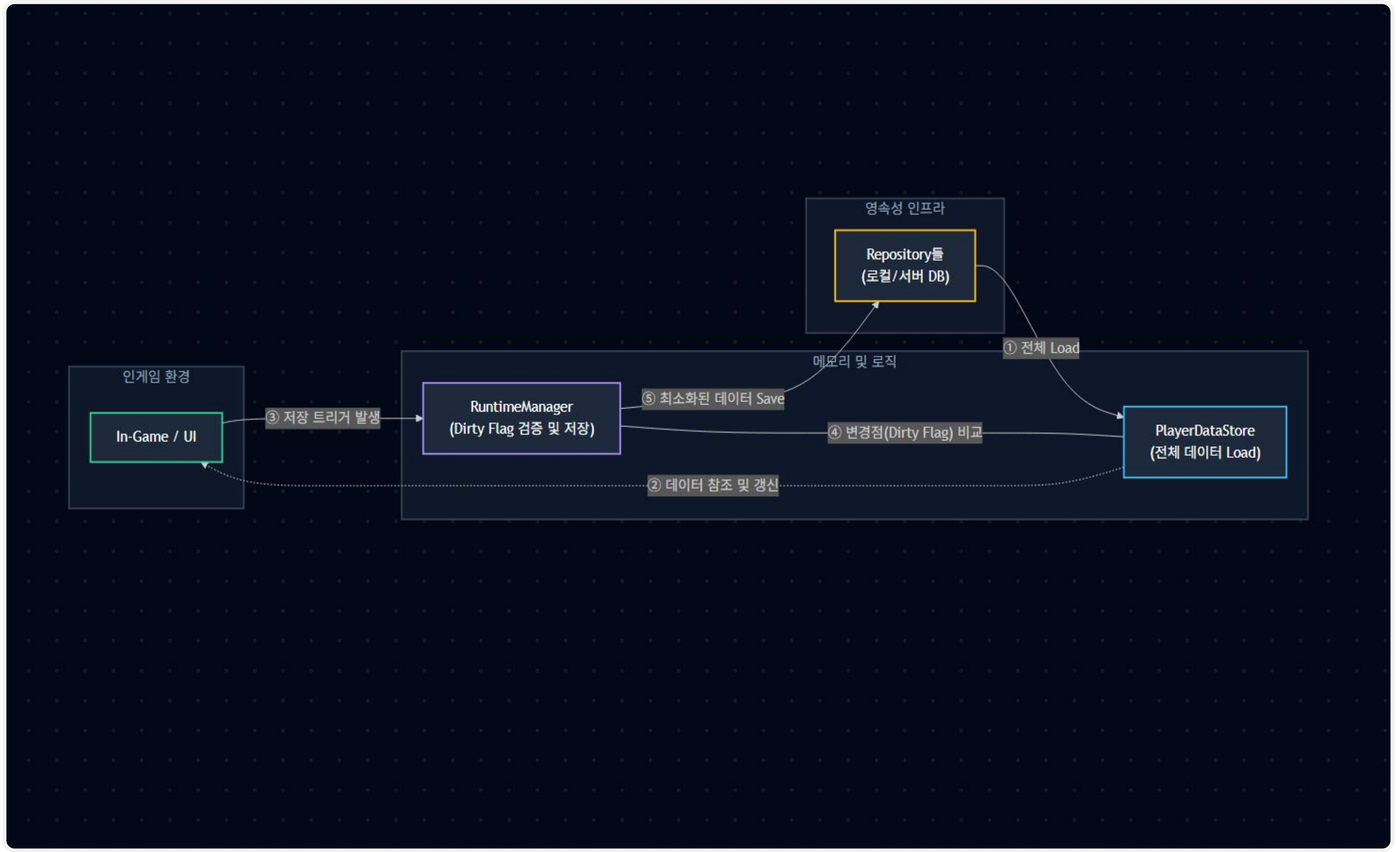
중앙 데이터 관리 구조 >>>

PlayerDataStore 로드 및 사용

RuntimeManager 저장

세부 설명

- PlayerDataStorea에서 Repository들의 데이터들을 모두 Load를 담당
- RuntimeManager에서 데이터 수정 및 저장 담당
  - 저장하기 전에 PlayerDataStore 데이터를 비교하여 검증
  - Dirty Flag로 저장 최소화



PlayerDataStore 및 RuntimeManager 도식화

인벤토리 시스템 >>>

세부 설명

- Dictionayr<List> 로 각 아이템 카테고리 별 구조
- 아이템 정렬 및 필터링, 합치기 기능 구현
- Outgame Item 중 드래그 앤 드랍으로 Ingame으로 가져가는 아이템만 Snapshot으로 넘겨서 데이터 경량화

이벤트 기반 UI 동기화 >>>

세부 설명

- 이벤트 기반으로 UI 동기화 최소화
- 캔버스 분리로 UI 최적화
- Outgame 인벤토리 UI와 Ingame 인벤토리 UI 분리 구현
- 백그라운드의 인벤토리 데이터(Model)와 화면에 렌더링되는 UI(View)를 분리

계층적 아이템 클래스 >>>

세부 설명

- 장비, 소비재, 재료 등 아이템의 특성에 맞춰 계층적 클래스(Class) 및 인터페이스 구조 설계
- 구글 SpreadSheet에 아이템 데이터만 추가하면 아이템 추가 가능하게 구현

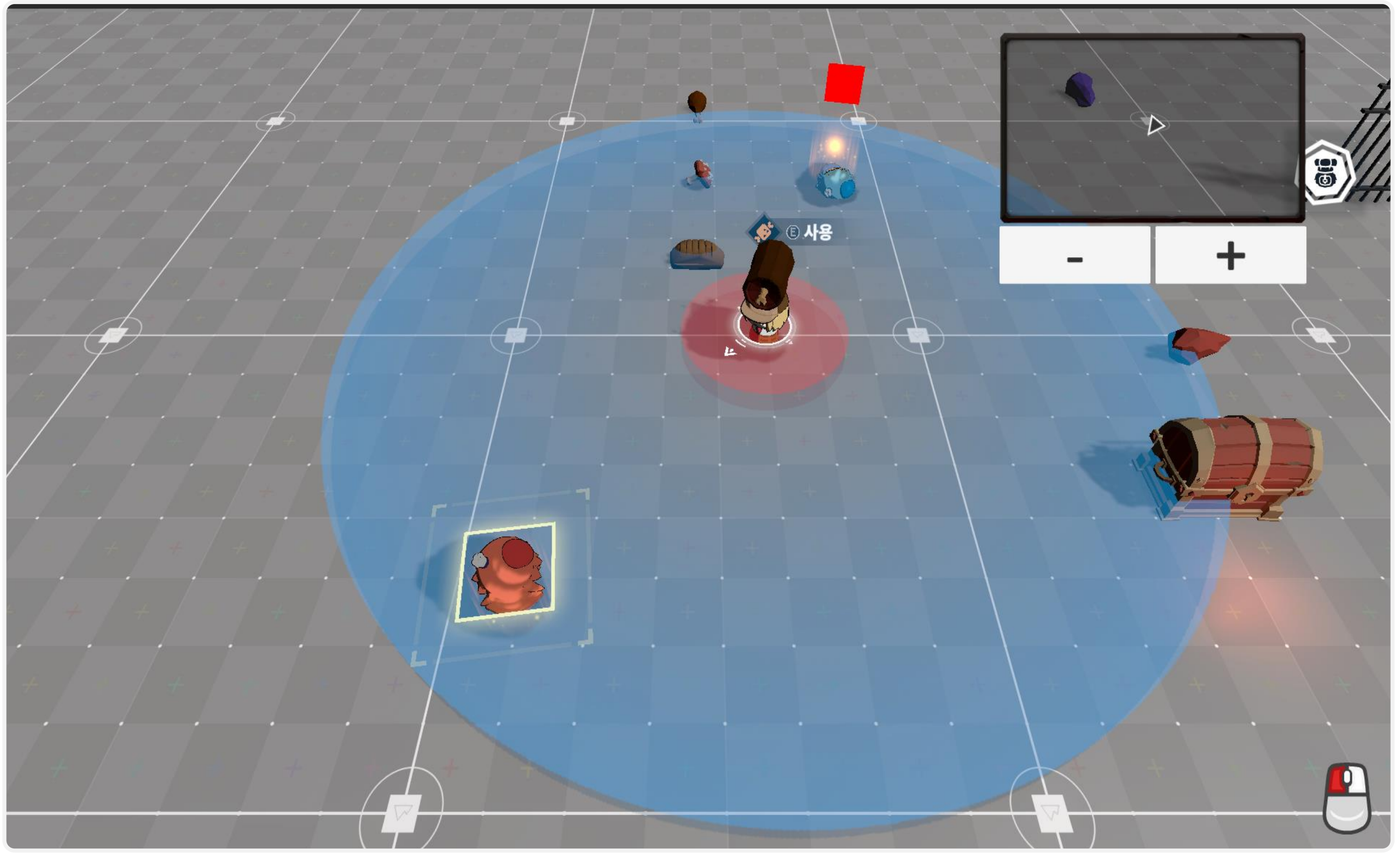
상호작용 시스템 >>>

Interface 기반의 유연한 설계

상황별 동적 상호작용 판정 및 예외 처리

세부 설명

- IInteractable 인터페이스 활용
- Context Interact & Physics Interact 분리
  - 들기 던지기 : Physics
  - 기타 상호작용 : Context
- 컴포넌트 구조 -> 원하는 상호작용에 맞게 컴포넌트 추가



상호작용 테스트 용 Scene

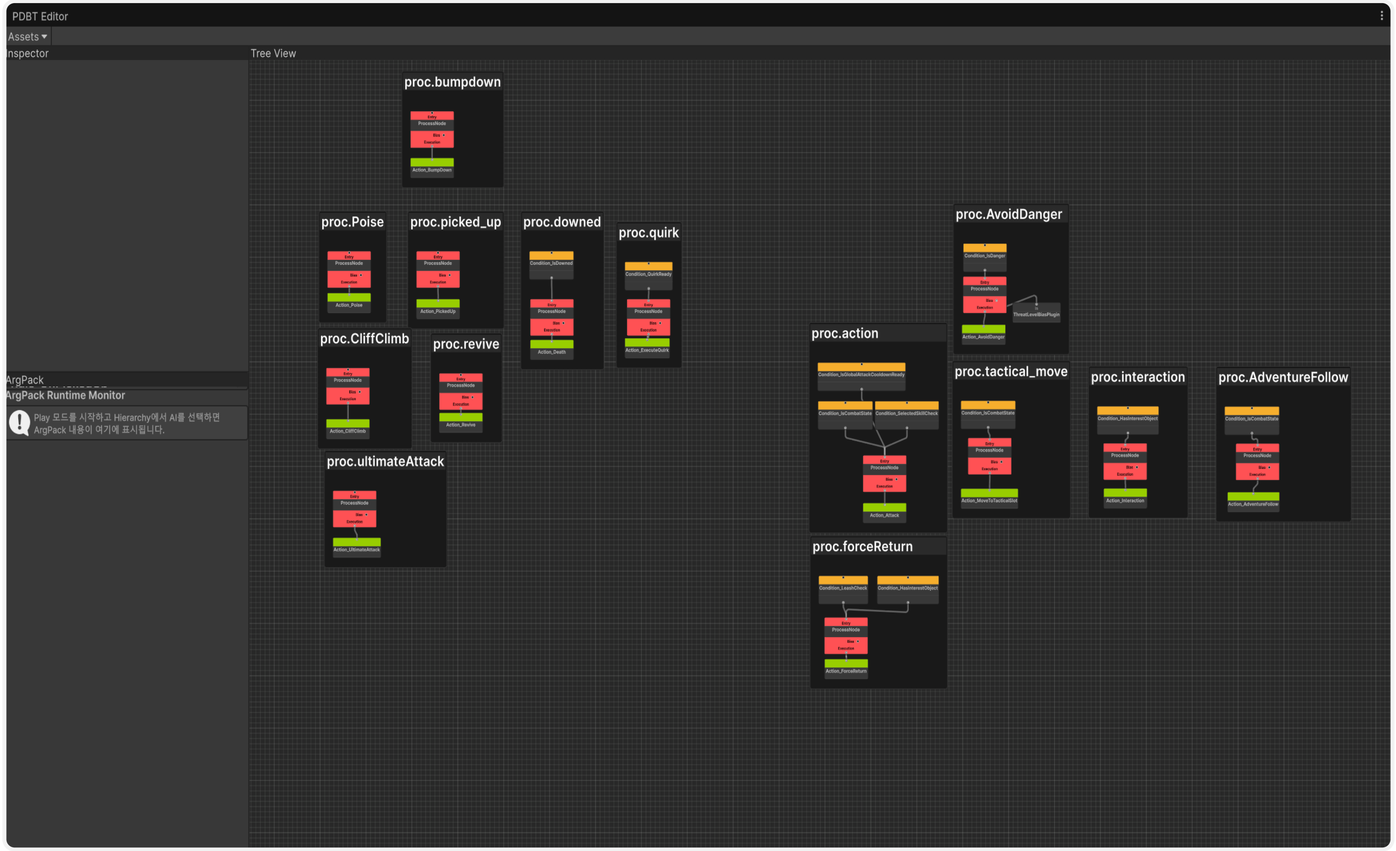
Behavior Tree 비주얼라이징 >>>

Unity Editor 스크립팅 기반 자체 툴 개발

기획자 협업 효율 및 AI 디버깅 환경 개선

세부 설명

- 노드 기반 시각화 에디터 구축
  - 기획자가 원하는 AI를 위한 새로운 형태의 BT 구현
  - UniTask를 활용해 Tick 주기 설정
- 런타임 디버깅 및 협업 고도화
  - 활성화된 노드 하이라이트



내부 AI 툴인 PDBT

Unity New Input System

세부 설명

- 확장성을 위해 레거시 Input Manager 대신 패키지 기반의 New Input System 전면 도입
- 키 바인딩 및 다양한 입력 디바이스(키보드, 마우스, 터치)를 일관된 이벤트로 처리

크로스 플랫폼 조작 대응

세부 설명

- 하나의 InputManager 클래스에서 PC(마우스/키보드)와 모바일(가상 조이스틱/터치) 입력을 추상화
- 빌드 타겟 또는 런타임 환경을 감지하여 알맞은 입력 UI 및 컨트롤러 로직 자동 전환

UI Input & Player Input 분리

세부 설명

- 인게임 캐릭터 조작 입력과 UI(인벤토리, 팝업 등) 조작 입력을 레이어로 완벽히 분리
- UI 팝업 시 플레이어 조작 차단 등 상태에 따른 Input 활성/비활성 제어 로직 구현

렌더링 및 드로우콜 최적화 >>>

세부 설명

- 정적/동적 배칭(Batching) 적용 및 URP(Universal Render Pipeline) 기반의 셰이더 경량화
- 캔버스(Canvas) 분리를 통해 UI 렌더링에 의한 드로우콜 스파이크 억제

메모리 및 병목 현상 개선 >>>

세부 설명

- Unity Profiler 및 Memory Profiler를 활용하여 매 프레임 발생하는 GC Alloc 추적 및 제거
- 빈번하게 생성/파괴되는 투사체, 이펙트, UI 요소에 오브젝트 풀링(Object Pooling) 전면 적용

물리 연산 최적화 >>>

세부 설명

- 불필요한 Rigidbody 컴포넌트 제거 및 충돌 매트릭스(Collision Matrix) 최적화
- 프레임 단위의 레이캐스트 연산을 비동기 또는 특정 프레임 간격으로 분산 처리(Time-Slicing

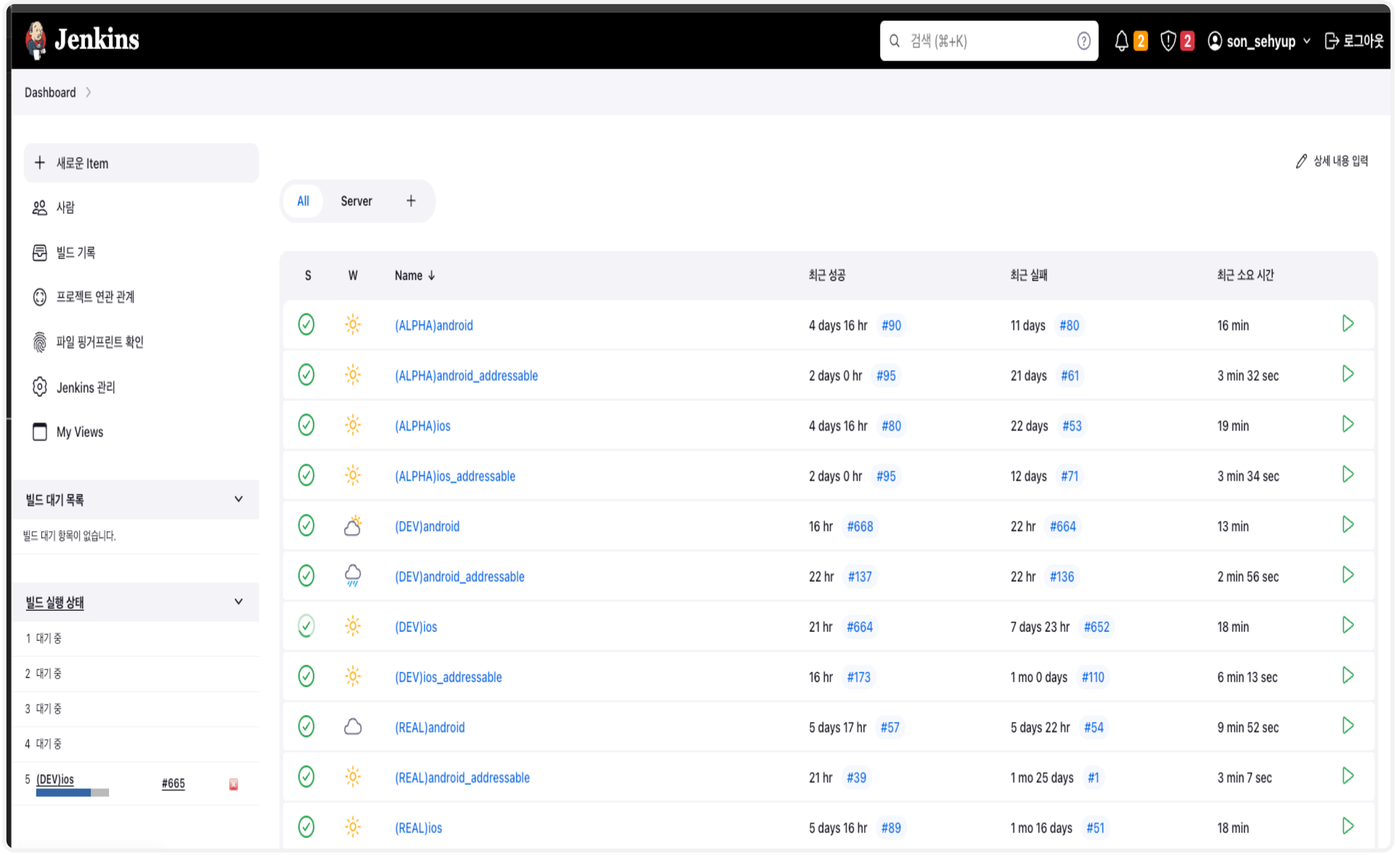
Jenkins 기반 CI/CD 파이프라인 구축>>>

Github 연동 branch 연결

시간 Trigger 및 MCP 연동

세부 설명

- 자동화된 빌드 환경 구축
  - 플랫폼별 빌드 프로세스 분리
  - 테스트 주기 가속화
- 효율적인 워크플로우 통합
  - 오전 9시에 빌드 되도록 세팅
  - 빌드 성공/실패 여부 Slack에 MCP로 요약 정보 전송



Jenkins 예시 이미지



Potion Maker

프로젝트 기간 | 2025-07-11 - 2025-08-21

프로젝트 인원 | 5 인

설명 | 포션을 제조하고 판매하며, 점차 포션 상점을 확장해 나가는
멀티플레이 협동 시뮬레이션 게임

담당 역할 | 게임 클라이언트 개발자, 모델링, 캐릭터 애니메이션

Github | https://github.com/Project-PotionMaker/PotionMaker

영상 | https://youtu.be/wG9I-TZsLJs

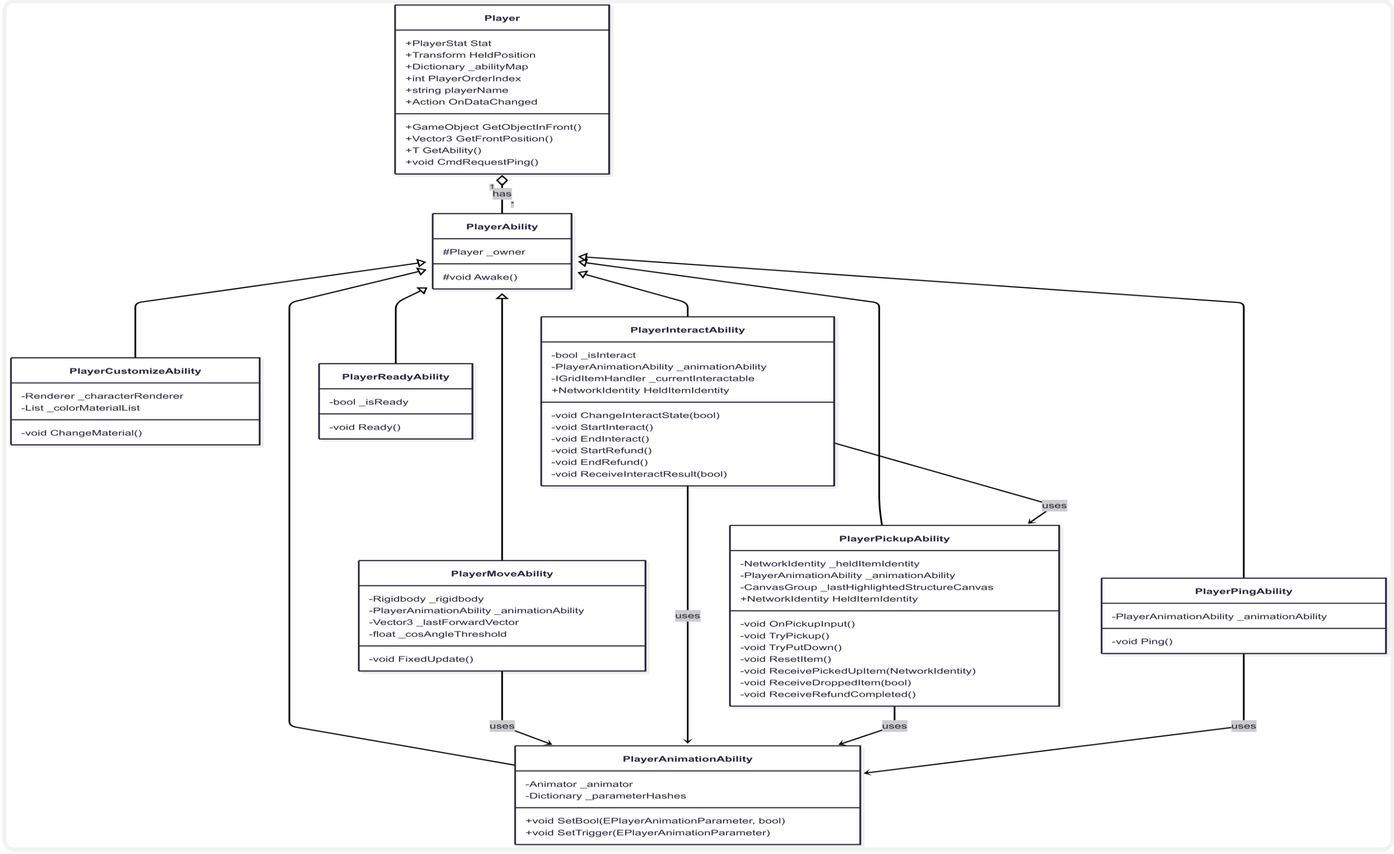
플레이어 시스템 >>>

컴포넌트 기반 아키텍처

개별 기능 어빌리티

세부 설명

- 플레이어의 모든 기능을 ‘Ability’로 분리
- 본인 캐릭터만 조작하도록 Input 관리
- ‘Player’ 클래스는 Ability들을 관리하는 허브(Hub) 역할만 수행
- 각 기능 로직을 개별 클래스로 구현
  - 유지보수 및 확장성



Player 컴포넌트 기반 아키텍처 도식화

플레이어 색 변경 >>>

온라인 Player Index – 서버에 입장한 순서

포토샵을 이용한 Texture 제작

세부 설명

- 온라인에서의 플레이어 인덱스를 확인
- 인덱스에 따른 플레이어 색을 변경하도록 구현
  - 미리 제작한 Material을 대입



인 게임 플레이어 색상

Player 및 NPC Animation 제작 »»

Blender MCP

AI Python 코드 제작

세부 설명

- 휴머노이드 타입이 아닌 리깅 구조
- Cursor와 Claude에서 MCP를 통해 Blender에 연결
- MCP를 통해 애니메이션 생성 요청
  - 자세하게 설명하여 요청
- 난해한 애니메이션은 Python Script 제작



Alpha 발표 PPT 내용

New Input System

세부 설명

- 키보드와 게임패드를 모두 지원하기 위해 Unity의 new input system을 선택
- Key Mapping을 구현하기 위해 선택
- InputManager를 통해 Input을 받고, 이벤트 방식으로 구현해서 필요한 곳에서 구독하는 방식으로 사용

Runtime Input Mapping

세부 설명

- 플레이어가 직접 키 세팅할 수 있도록 지원
- 주 키와 보조 키 설정 가능
- 다양한 게임패드를 지원하도록 구현
  - 게임패드마다의 아이콘으로 표현

UI Input

세부 설명

- Player Input과 UI Input을 분리
- UI 팝업창 생성 시 UI 모드로 변경
  - Player는 동작 안 하게 설정
- 게임패드를 위한 키로 UI 조작 가능하게 구현

도감 시스템 >>>

Addressable 이미지 로드

구글 Data Sheet 연동

세부 설명

- Addressable에서 이미지를 로드하여 사용
  - 로딩 씬에서 미리 로드 후 사용
- 구글 드라이브에서 Data Sheet와 연동
  - Data Sheet에서 추가하면 알아서 도감도 추가



인 게임 도감 UI

상점 외벽 모델링 >>>

Meshy AI 활용

Tripo AI Texture 제작

세부 설명

- Perplexity를 통해 레퍼런스 이미지 생성
- Meshy AI를 통해 외벽 모델 생성
  - 레퍼런스 이미지 사용
- Blender 모델링
  - Meshy AI를 통해 생성한 모델 참조
- Tripo AI를 이용해 Texture 생성
  - 레퍼런스 이미지 사용



Alpha 발표 PPT 내용

Factory 패턴 >>>

세부 설명

- Network 생성과 삭제를 위한 Factory 패턴 구현
- 상속받아서 사용가능한 BaseNetworkFactory를 구현하여, 다른 개발자들이 사용할 때 편하게 수정할 수 있도록 구현
- FactoryLogic 클래스를 따로 만들어서 내부 구현을 다른 클래스로 구현

Mirror >>>

세부 설명

- Unity의 Mirror를 사용해 Network 동기화 생성 구현
- 생성과 삭제는 무조건 Server에서만 일어나도록 구현
- Addressable을 사용하여 동적으로 로드한 에셋을 사용하도록 구현
  - Enum으로 받아서 string으로 변환 후 사용

Object Pooling >>>

세부 설명

- Mirror의 NetworkClient의 Custom Spawn Handler를 통해 구현
- 앞서 설명한 FactoryLogic에서 구현
- 내부적으로 Queue를 통해 구현
- Enum으로 타입을 받아서 생성하도록 설정
- 풀링에서 나갈 때 오브젝트 별로 타입을 등록, 들어올 때 해당 오브젝트로 맞는 타입의 Queue로 반환



SKYFALL : Resistance

프로젝트 기간 | 2025-05-15 - 2025-06-05

프로젝트 인원 | 4 인

설명 | 공중에서 낙하하며, 빠른 속도감을 느끼며
전투를 하는 TPS 핵앤슬래시

담당 역할 | 환경 프로그래밍, Effect 프로그래밍, 상호작용 구현

Github | https://github.com/Project-FallTPS/SkyFall-Resistance_Client

영상 | https://youtu.be/eVyqCoeKIqA?si=a9sHUSH5D1a-yb4x

낙하 전투 환경

커스텀 Shader – 구름, 잔해물, 자기장

동적 환경 구성

세부 설명

- 커스텀 Shader를 통해 환경 구성
  - 구름, 잔해물(낙하물), 자기장 영역
- 외부 에셋 커스터마이징
  - 외부에서 가져온 에셋을 게임의 분위기에 맞게 수정



인 게임 Fall Scene

낙하 전투 환경 구성 >>>

세부 설명

- 실제 플레이어는 떨어지지 않지만 주변 환경으로 떨어지는 듯한 착시 효과 구현
  - 배경, 잔해물 등
- 주변 구름 및 보스전을 진행할 섬을 위쪽 방향을 향하도록 설정
- 구름은 Layer를 나누어서 플레이어한테 멀수록 천천히 올라가도록 설정

동적 환경 길이 수정 >>>

세부 설명

- 전체 페이즈의 시간을 계산하여 동적으로 낙하하는 환경의 길이 수정
- 종단 속력으로 떨어져서 일정한 속력으로 떨어진다고 가정하고 길이 = 시간 X 속력으로 계산
- 잔해물들은 생성시 힘을 줘서 다양한 방향으로 향하도록 설정
- 특정 구역 안에 들어오면 속력을 줄여서 같이 떨어지고 있다는 느낌을 구현

VFX >>>

세부 설명

- 잔해물들의 궤적을 불 VFX를 이용하여 그리면서 떨어지도록 외부 VFX 수정
- 자기장 영역을 직접 Shader Graph를 통해 구현
- 자기장을 이용하여 플레이어를 특정 구역 내에서만 움직이게 하여 보이는 외부 환경의 상태를 유지

Bezier 곡선 총알 충돌 오류 >>>

세부 설명

- 보스 몬스터의 패턴으로 직사와 곡사 탄환을 구현
  - Behavior tree로 구현
- 곡사 패턴은 Bezier 곡선으로 구현
- Bezier 곡선을 사용한 총알이 벽을 뚫고 지나가는 상황 발생

보간 곡선 VS 근사 곡선 >>>

세부 설명

- 근사 곡선은 처음과 마지막 제어점만 통과, 중간 제어점들은 통과하지 않음
  - Bezier 곡선은 대표적인 근사 곡선
  - 점의 위치들만 필요
- 보간 곡선은 모든 점들과 그 점에서의 접선 기울기를 정확히 통과하고 따른다.
  - 각 점 + 점마다 기울기 필요

Hermite 곡선 >>>

세부 설명

- 보간 곡선 중 하나인 Hermite 곡선을 사용
- 본인 위치(출발점) + 플레이어 위치(도착점) + 장애물 주변 위치(중간점)
  - 세 점으로 벡터를 만들어서 접선의 기울기 결정
- 보스전의 난이도 상승 및 정확한 궤적 구현

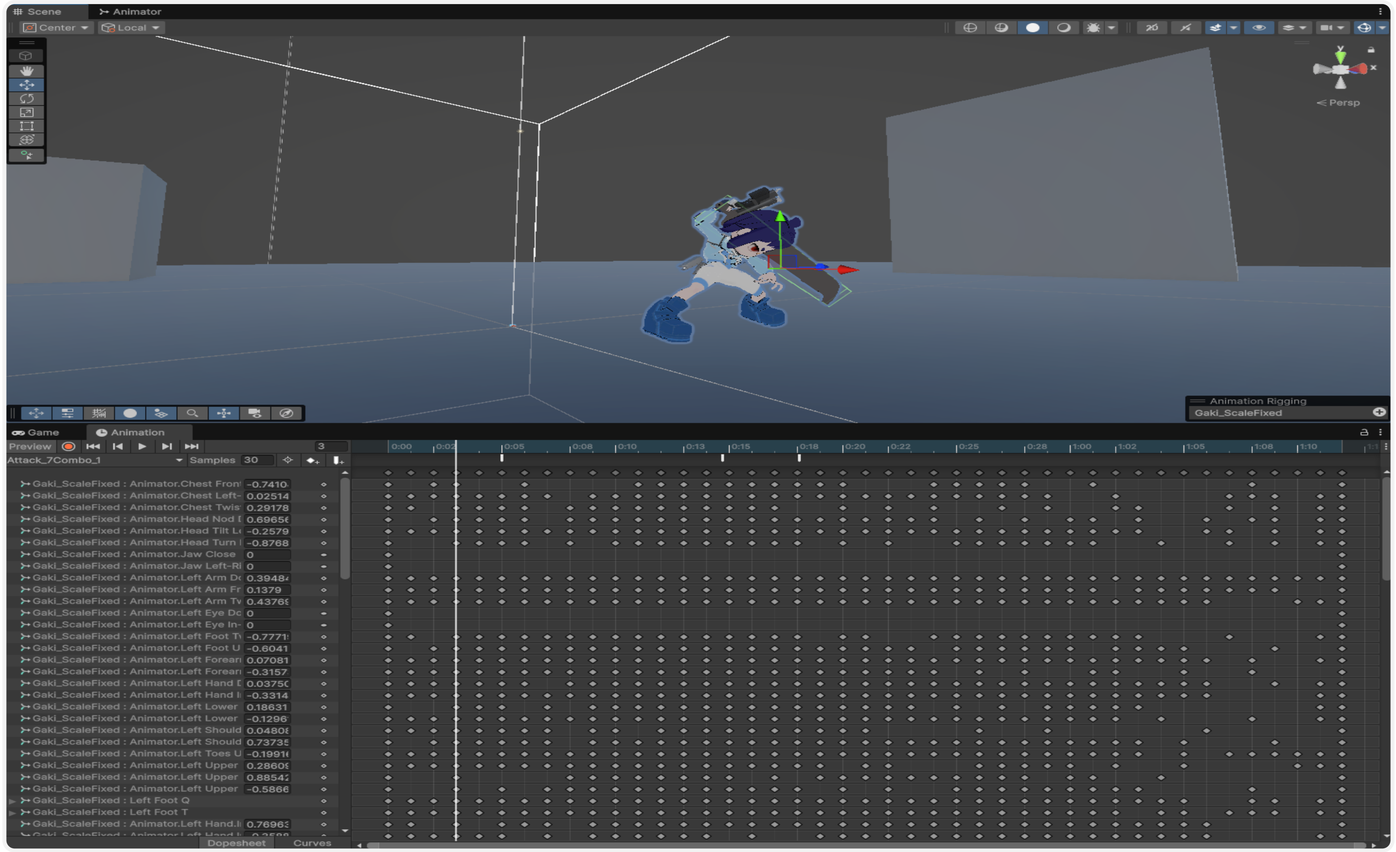
플레이어 애니메이션 수정 >>>

모델 변경에 의한 애니메이션 붕괴

프레임 단위별 수정

세부 설명

- 플레이어의 모델을 변경하면서 애니메이션에 의해 모델이 겹치는 상황 발생
- 애니메이션 클립을 프레임 단위로 수정
  - 게임의 퀄리티 유지



Player Animation 수정 작업 화면

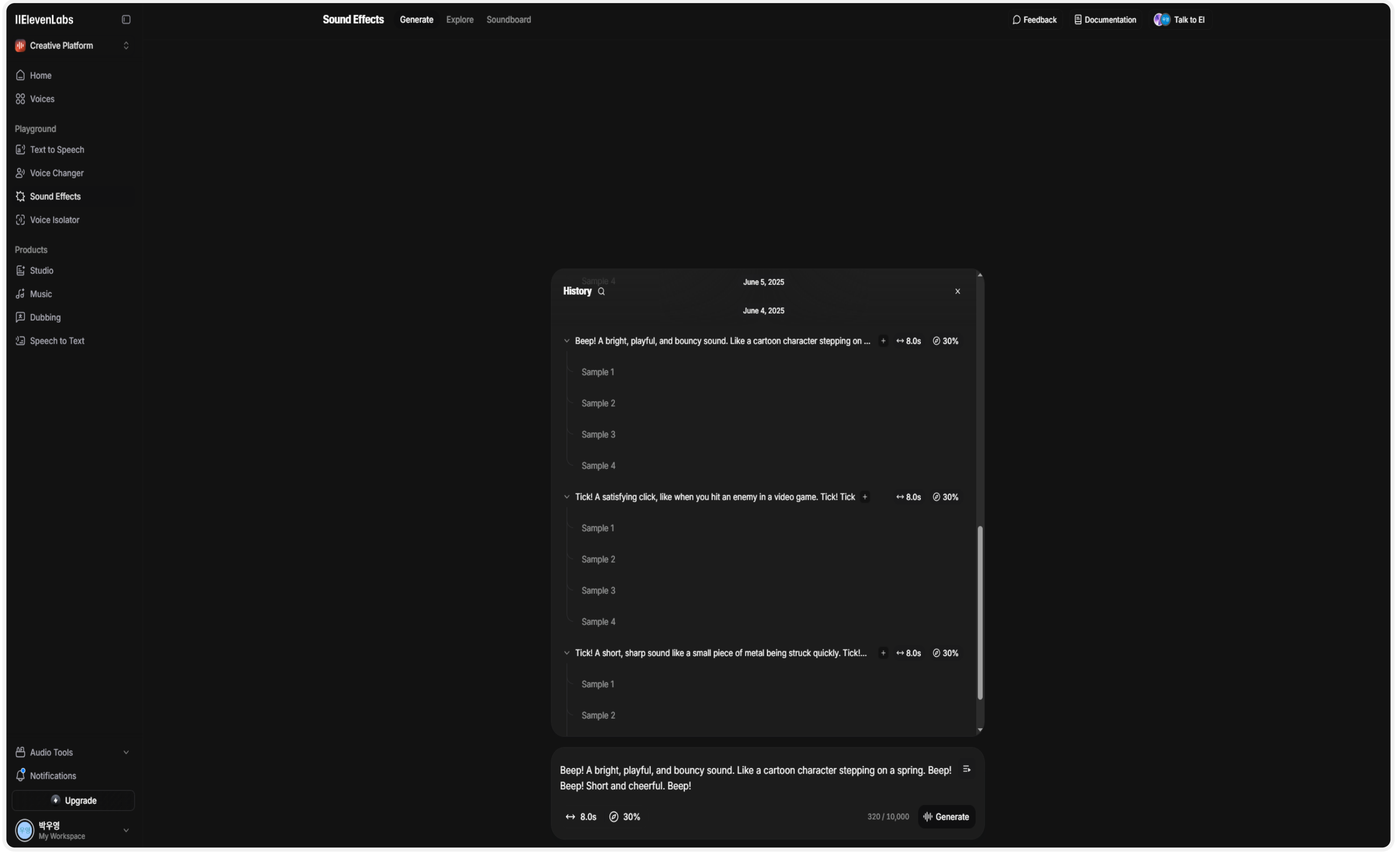
SFX

Pixabay – 무료 사운드 탐색

ElevenLabs – 효과음 생성

세부 설명

- Pixabay를 통해 BGM 및 사운드 이펙트 추가
- EleveLabs를 통해 부족한 사운드 생성
- 사운드의 타이밍을 위해 사운드 편집 및 재생 타이밍 조절



ElevenLabs Sound Effects 제작 화면



Last Flame Keepers

프로젝트 기간 | 2025-04-02 - 2025-04-21

프로젝트 인원 | 4 인

설명 | 어둠 속 마지막 “불”을 지키고 성장시키며, 맵 전체를 밝혀야 승리하는 생존형 디펜스 시뮬레이션 게임

담당 역할 | 건물 시스템, UI, 시스템 연동형 구조 설계

Github | https://github.com/sangjin256/lastflamekeepers (비공개)

영상 | https://youtu.be/-JJ-dlSisqs

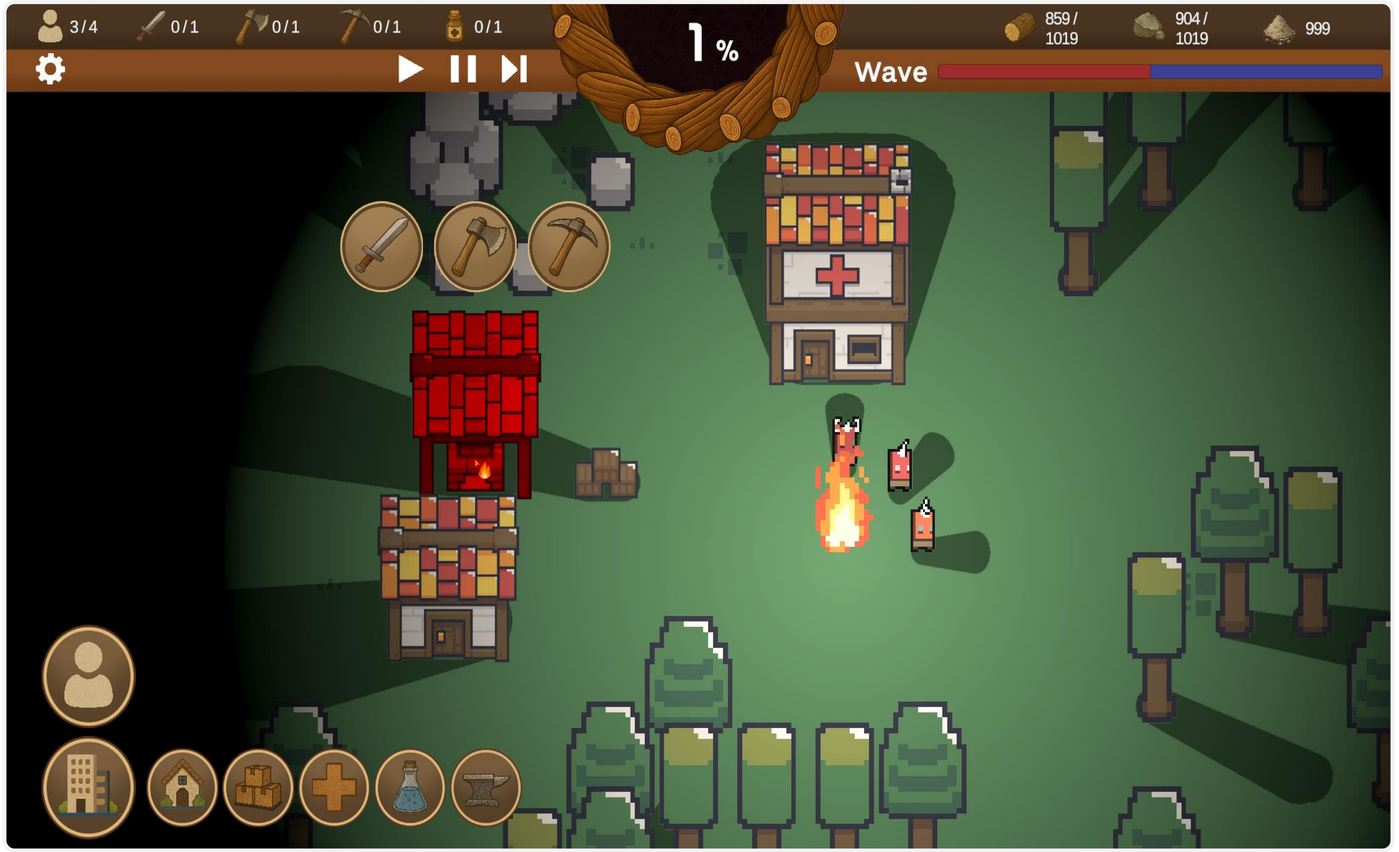
건물 시스템

건물 설치 및 활성화/비활성화

건물 업그레이드

세부 설명

- 건물 설치 시 Collider를 통해 설치 가능한지 확인
- 불 영역 확장 및 수축에 의한 건물 활성화/비활성화 구현
  - 우선순위 큐를 통해 구현
- 건물 별 업그레이드 효과 구현



인 게임 건물 설치 판정 화면

Unity Asset 우선순위 큐 >>>

세부 설명

- Unity Asset Store에 있는 우선순위 큐 는 우선순위를 비교하는 데이터 타입이 int로 고정
- 건물이 불로부터의 거리를 담기 위해서는 float가 필요

우선순위 큐 수정 >>>

세부 설명

- 우선순위 큐 Asset을 직접 수정하여 Generic 타입으로 변형
- PriorityQueue<Telement,Tpriority>로 변형
- 다른 개발자가 필요한 곳에 사용할 수 있도록 개발

우선순위 큐 사용 >>>

세부 설명

- 건물 간 거리 우선순위 정렬
- 불의 영역 크기가 변할 때 우선순위 큐에서 제일 위쪽의 건물을 꺼내서 거리를 확인
  - List보다 최적화를 위해
  - 반복문을 돌며 가능한 부분까지 활성/비활성 작업

UI 구현

UI – Button 로직 구현

UI Tweening

세부 설명

- for 문을 통해 버튼 onClick 메서드 연결 시 for 문의 int 값을 직접 사용하지 말고, 내부의 새로운 변수에 대입하여 사용
  - 그렇지 않으면 마지막 int 값을 버튼에 할당
- DOTween을 사용하여 UI Animation을 구현



인 게임 유닛 선택 화면



Billiards : 3 Ball

프로젝트 기간 | 2022-09-01 - 2022-12-21

프로젝트 인원 | 3 인

설명 | Python을 이용한 당구(3구) 게임

담당 역할 | PM, 전체 프로그래밍, 물리 엔진 구현

Github | https://github.com/wooing26/pygame_billiards

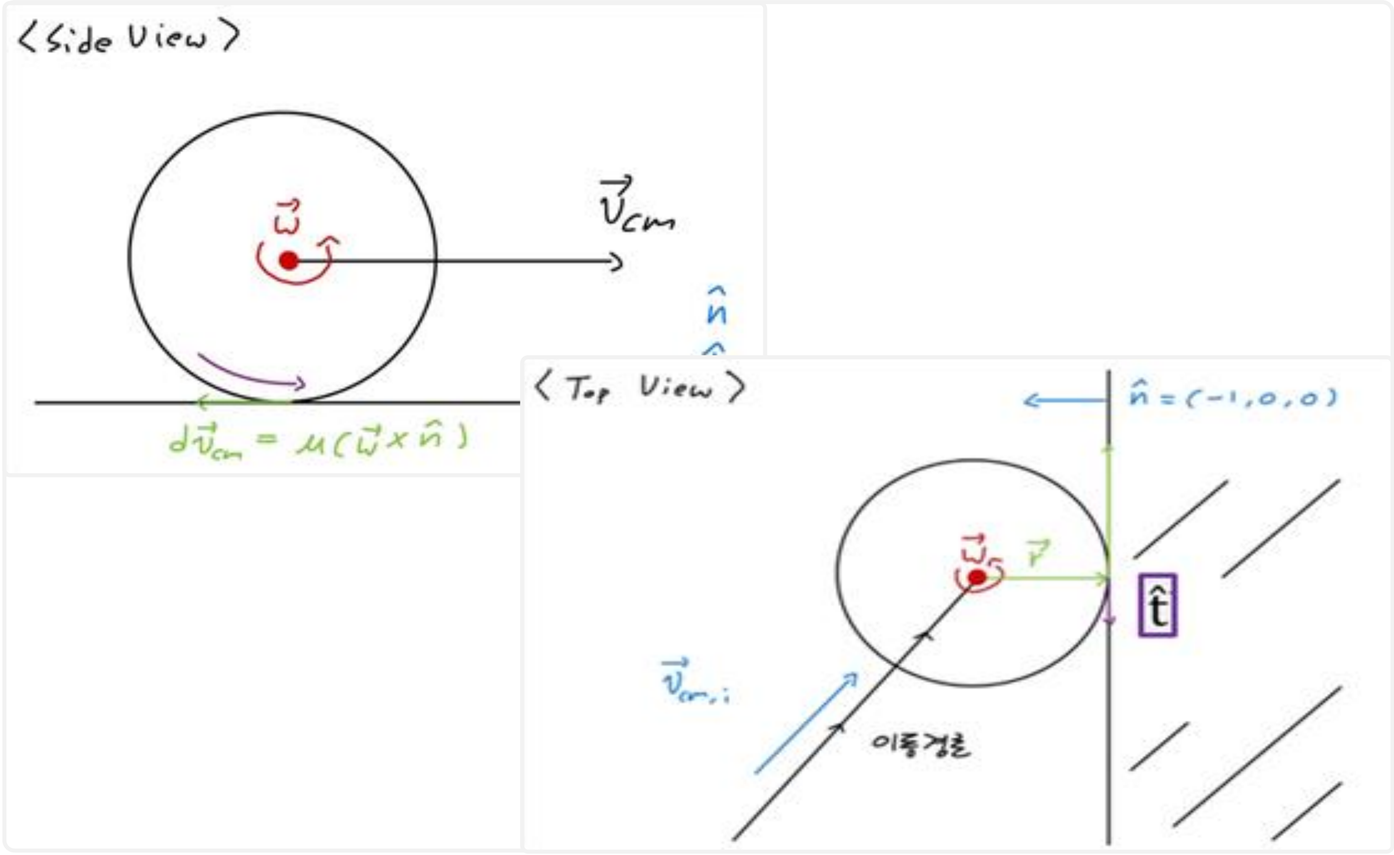
물리 시스템 구현 >>>

공의 속도, 회전, 마찰

공의 충돌 – 벽, 다른 공

세부 설명

- 실제 공의 움직임을 물리학적으로 분석
- 실제 상수 값들을 대입하여 구현
  - 공의 질량, 부피, 밀도
  - 당구대의 마찰 계수
  - 당구공의 반발 계수
- 충돌 후 속도, 스핀을 공의 반발계수를 사용하여 구현



공의 충돌 및 움직임 계산 이미지

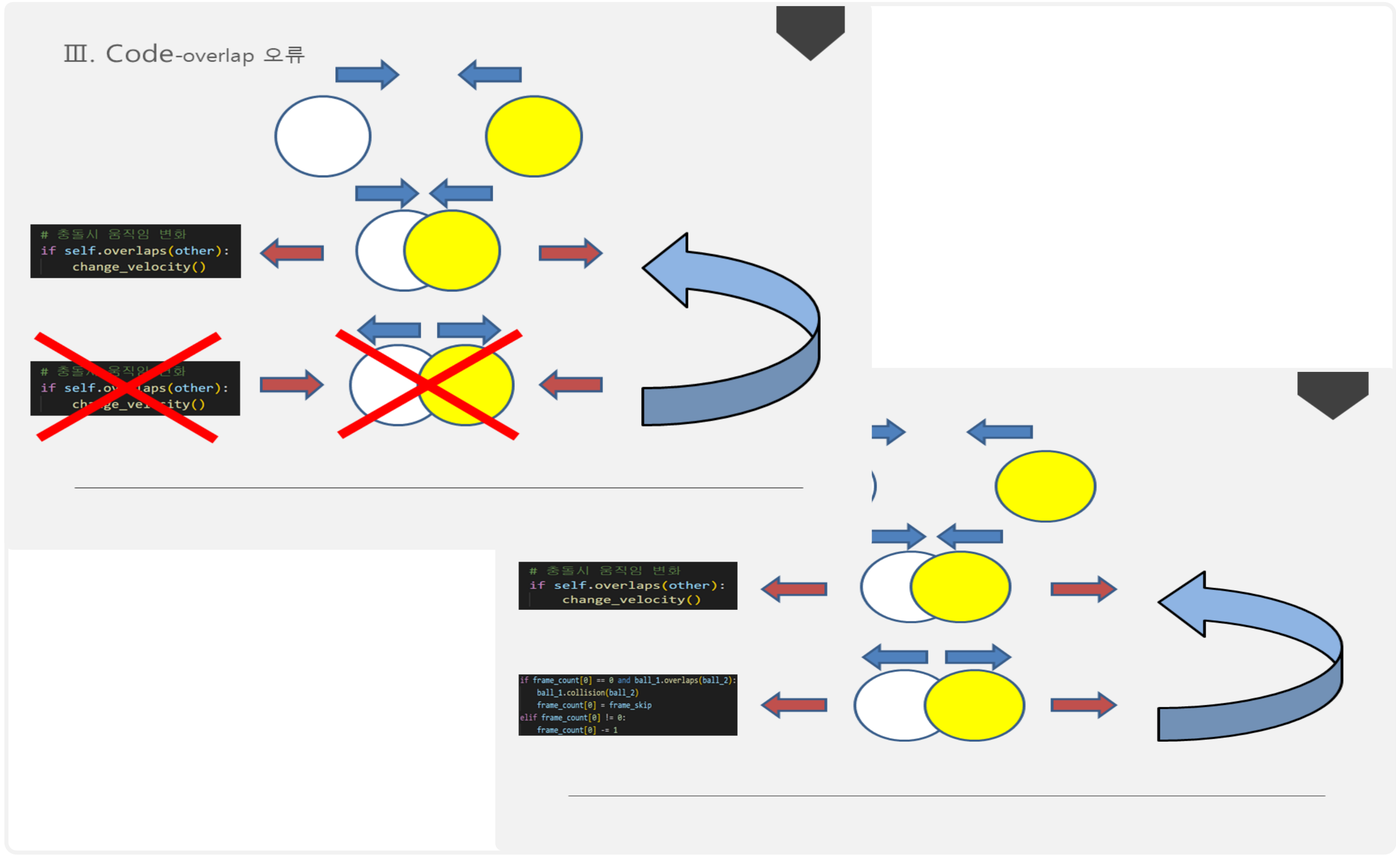
Overlap 오류

공과 공이 충돌 후 발생

프레임별 로직 계산 에러

세부 설명

- 공과 공이 충돌한 후 겹친 상태로 함께 이동하는 에러가 발생
- 충돌 후 속도를 계산하는 로직 이후 다음 프레임에 아직 겹쳐 있으면 또 충돌 로직에 의해 반대로 이동
- 충돌 이후 몇 프레임을 건너 뜀으로 해결



해당 Error 해결 과정 PPT

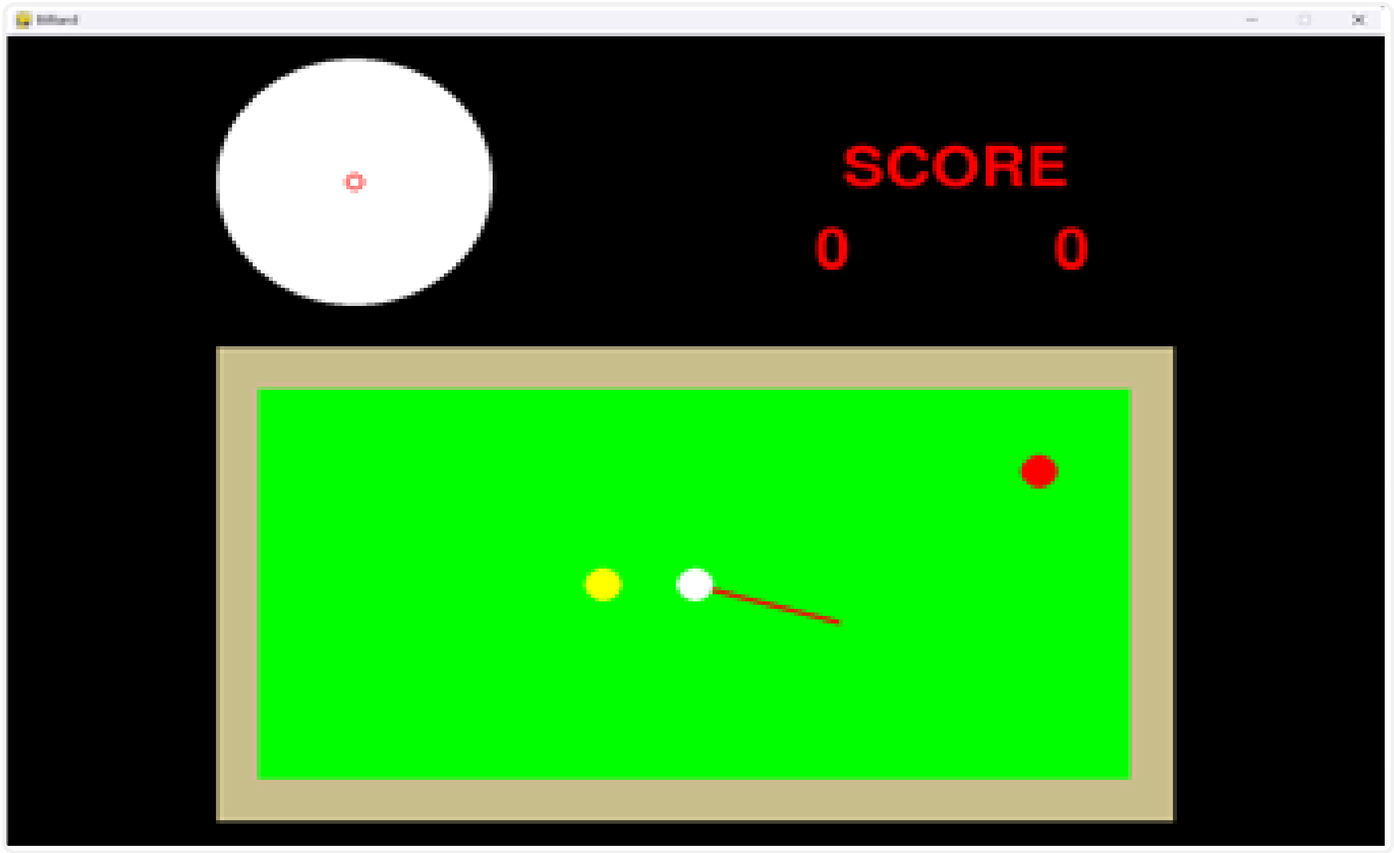
스핀 입력 >>>

바라보고 있는 방향의 단면 공 UI

Numpy arccos 메서드 범위 문제

세부 설명

- 실제 당구에서 바라보고 있는 방향으로 치는 느낌이 나게 UI 설계
- 왼쪽 위에 공에서 스핀 입력
- Numpy arrcos으로 스핀 입력 값을 계산
  - arccos의 범위에 의해 원하는 값이 안 나옴
- 회전 벡터의 크기 정규화 후 cos 및 sin을 통해 각도 계산 없이 스핀 방향 처리



인 게임 화면